

Assignment-defense questions

Code-level questions for each assignment. Expect “open your file and explain line X.” Open the actual file when answering and give the line number back.

Exercise set 1

Files: - assignments/exercise-set-1/exercise_1_computational_finance (2).py - assignments/exercise-set-1/exercise_4_computational_finance (2).py

Exercise 1 — implied volatility

1. Open `exercise_1_computational_finance (2).py` and explain lines 13-14 of `C_bs`: what does the guard `if sigma <= 0 or T <= 0` return, and why is $\max(S_0 - K \cdot e^{-rT}, 0)$ the right fallback value rather than 0? **A.** The guard returns the option's intrinsic value in forward-discounted terms, $\max(S_0 - K e^{-rT}, 0)$. When $\sigma \rightarrow 0$ or $T \rightarrow 0$ the Black-Scholes formula divides by $\sigma \sqrt{T} \rightarrow 0$ (the `d1`, `d2` definitions on lines 15-16 are undefined), so I short-circuit. The right limiting value is not 0: as $\sigma \rightarrow 0$ the stock grows deterministically to $S_0 e^{rT}$, and the discounted payoff $e^{-rT} \max(S_0 e^{rT} - K, 0) = \max(S_0 - K e^{-rT}, 0)$. That is the no-arbitrage lower bound of a call, so the function stays continuous and never returns a value below the bound. Returning plain 0 would wrongly price a deep-ITM call at zero.
2. In `implied_volatility_bisection` (lines 19-41), what does the function return on line 29, and under what condition? Explain in terms of the sign of `f_low * f_high`. **A.** Line 29 returns `np.nan` when `f_low * f_high > 0`, i.e. when $C_{bs}(\sigma_{low}) - C_{mkt}$ and $C_{bs}(\sigma_{high}) - C_{mkt}$ have the same sign. Bisection needs a sign change to guarantee a root in $[\sigma_{low}, \sigma_{high}]$ (intermediate value theorem); if both endpoints sit on the same side, the market price lies outside the achievable BS range on $[1e-6, 5.0]$ (e.g. a price violating no-arbitrage bounds), so no implied vol exists in the bracket and returning NaN flags that path rather than producing a garbage number.
3. Line 34 has a compound stopping test `abs(f_mid) < tol or (sigma_high - sigma_low) < tol`. Which of the two conditions is in *price* units and which is in *vol* units? Why can the loop exit on either? **A.** `abs(f_mid) < tol` is in price (option-value) units — `f_mid = C_bs(σ_mid) - C_mkt` is a price residual. `(sigma_high - sigma_low) < tol` is in volatility units — the width of the bracket. Either is a valid termination: the first says the price is matched to tolerance, the second says the vol is pinned to tolerance. Both must hold eventually since the bracket halves each step; including both protects against the flat-vega regime where the price residual stays large per unit of σ but the interval has already collapsed.
4. Vega (lines 43-49) returns $S_0 \cdot \text{np.sqrt}(T) \cdot \text{norm.pdf}(d1)$. Why is this exactly $\partial C_{BS} / \partial \sigma$, and why does it appear in the Newton denominator on line 66? **A.** Differentiating $C_{BS} = S_0 N(d1) - K e^{-rT} N(d2)$ w.r.t. σ , the explicit σ -terms via `d1`, `d2` cancel by the identity $S_0 \phi(d1) = K e^{-rT} \phi(d2)$ (where $\phi = \text{norm.pdf}$), leaving only $\partial C / \partial \sigma = S_0 \phi(d1) \partial d1 / \partial \sigma - [\text{same cancels}]$, and $\partial d1 / \partial \sigma$'s clean part gives $S_0 \sqrt{T} \phi(d1)$. So Vega is $\partial C_{BS} / \partial \sigma$. Newton solves $f(\sigma) = C_{BS}(\sigma) - C_{mkt} = 0$, and $f'(\sigma) = \text{Vega}$, so line 66 $\sigma -= f/f'$ is $\sigma -= \text{diff}/\text{vega}$.
5. Explain line 58: `sigma_initial = np.sqrt((1/T)*(2*np.abs(np.log(S0/K)+ r*T)))`. What is this initial guess and what does the report claim it guarantees? Where in the lectures does it come from? **A.** This is the Brenner-Subrahmanyam / Manaster-Koehler starting point $\sigma_0 = \sqrt{(2/T) \cdot |\ln(S_0/K) + rT|}$. Manaster-Koehler proved this is the σ at which Vega is maximal (the inflection of C_{BS} in σ), which puts Newton on the side of the curve where it converges *monotonically and globally* to the implied vol — that is the guarantee the report cites. It comes from the Lecture 3 root-finding slides on Newton for implied vol.
6. On line 66, `sigma -= diff/vega`. If vega were ~ 0 (deep OTM, short T), what would happen numerically, and which method (bisection vs Newton) is safe against this? Which line shows the safe behaviour? **A.** If $\text{vega} \approx 0$, `diff/vega` blows up (division by near-zero) and Newton takes a huge, possibly diverging step or overflows. Bisection is safe because it never divides by a derivative — it only checks signs and halves the interval (lines 31-40); its convergence

depends only on the bracketing, not on Vega. The safe behaviour is on line 34's interval test and the halving on line 31.

7. Lines 94-96 build `IV_bisection` and `IV_Newton`. What column is passed as `C_mkt`, and why does that choice (mid vs bid/ask) directly feed your Exercise 1.3 discrepancy discussion? **A.** `C_mkt` is the 'Midpoint' column (line 94-95 zip over `df_calls['Midpoint']`). $\text{Mid} = (\text{bid} + \text{ask})/2$, so the inverted IV is a single point estimate sitting inside the bid-ask spread. Discrepancies versus the CSV reference IV (and the bid/ask one would get from the quoted spread) come from this choice: the bid-ask spread maps to an IV band, and using mid collapses it to one number — that band width, plus discretisation of strikes, is exactly the source of the 1.3 mismatch.
8. Roughly how many bisection iterations are needed to shrink the bracket $[1e-6, 5.0]$ below $\text{tol} = 1e-6$? Why is `max_iter = 200` (line 90) more than enough? **A.** The bracket has width ≈ 5 , and each step halves it, so I need $5/2^n < 1e-6 \Rightarrow 2^n > 5e6 \Rightarrow n > \log_2(5e6) \approx 22.3$, so about 23 iterations. `max_iter = 200` is $\sim 10\times$ that, so the iteration cap is never the binding constraint; the tolerance test on line 34 always fires first.

Exercise 4 — Monte Carlo basket call

9. Open `exercise_4_computational_finance (2).py`. Explain lines 25-26: how do $W1 = Z1$ and $W2 = \rho Z1 + \sqrt{1-\rho^2} Z2$ produce two Brownians with $dW1 dW2 = \rho dt$? Why is this the Cholesky factor of a 2×2 correlation matrix? **A.** $Z1, Z2$ are i.i.d. $N(0,1)$. Then $\text{Var}(W2) = \rho^2 \cdot 1 + (1-\rho^2) \cdot 1 = 1$, and $\text{Cov}(W1, W2) = E[Z1(\rho Z1 + \sqrt{1-\rho^2} Z2)] = \rho$, so $(W1, W2)$ are standard normals with correlation ρ . The map $[Z1, Z2] \mapsto [W1, W2]$ uses the lower-triangular matrix $\begin{bmatrix} 1 & 0 \\ \rho & \sqrt{1-\rho^2} \end{bmatrix}$, which is exactly the Cholesky factor L of the correlation matrix $\begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix} = L L^T$. Scaling by $\sqrt{T}$ (or working in increments) gives Brownians with $dW1 dW2 = \rho dt$.
10. Compare line 29 (drift $r - 0.5\sigma^2$, under Q) with line 65 (drift $r + 0.5\sigma_1^2$, under $QS1$). Why does the sign in front of the $\frac{1}{2}\sigma_1^2$ term flip, and which result from Exercise 3(c) does that come from? **A.** Under Q the asset has log-drift $r - \frac{1}{2}\sigma_1^2$ (Itô correction for the lognormal). Under the measure $QS1$ with numéraire $S1$, by Girsanov the Brownian driving $S1$ picks up a $+\sigma_1 dt$ drift shift, which turns $S1$'s log-drift into $r - \frac{1}{2}\sigma_1^2 + \sigma_1^2 = r + \frac{1}{2}\sigma_1^2$. That sign flip is exactly the change-of-numéraire / Radon-Nikodym result derived in 3(c): changing numéraire from the bank account to $S1$ adds $\sigma_1^2 T$ to $S1$'s exponent (and $\rho\sigma_1\sigma_2 T$ to $S2$'s, line 66).
11. Line 71 returns $V = S0[0] * \text{np.maximum}(AT-K, 0) / S1$ with **no** e^{-rT} factor, whereas line 35 (under Q) **does** discount. Derive why the discount factor is absent under $QS1$. **A.** Under Q , $V0 = e^{-rT} E^Q[\text{payoff}]$ — the bank account e^{rt} is the numéraire, hence the discount. Under $QS1$ the numéraire is $S1$ itself, so the pricing formula is $V0 = S1(0) E^{QS1}[\text{payoff} / S1(T)]$. The $1/S1$ on line 71 is the division by the terminal numéraire and the $S0[0]$ prefactor is $S1(0)$; there is no e^{-rT} because the deterministic bank-account discount has been replaced by the stochastic numéraire $S1$. The two estimators target the same $V0$.
12. What exactly do lines 38-39 (and 74-75) return, and why is `ddof=1` used in `np.std`? What would change if you used `ddof=0`? **A.** They return $V_{\text{approx}} = \text{mean}(V)$ (the MC price estimator) and $SE_{\text{approx}} = \text{std}(V, \text{ddof}=1) / \sqrt{M}$ (its standard error). `ddof=1` gives the unbiased sample variance (divide by $M-1$), which is the correct estimator of the population variance feeding the CLT-based SE. `ddof=0` divides by M , slightly underestimating the variance and hence the SE; for the large M here (10k-100k) the difference is negligible ($\sim \text{factor } \sqrt{M/(M-1)} \approx 1$), but `ddof=1` is the principled choice.
13. Lines 118-122 construct the $1/\sqrt{M}$ reference curves. What is the constant $c_Q = SEQ[0] * \text{np.sqrt}(\text{values}_M[0])$ doing, and why does plotting $c_Q / \text{np.sqrt}(M)$ test the CLT rate rather than assume it? **A.** $c_Q = SE(M0) \cdot \sqrt{M0}$ recovers the proportionality constant ($\approx$ the population std of the payoff) from the *first* data point only, anchoring the reference line $c_Q / \sqrt{M}$. Because the constant is fixed from one M and the rest of the curve is pure $1/\sqrt{M}$, overlaying it on the measured SEs is a genuine test: if the measured SEs fall on the line, the empirical decay rate really is $-1/2$. It tests the rate because only the level (not the slope) was fitted.

14. `np.random.seed(44)` is set on line 14 (Q) and line 50 (QS1). What is the consequence of re-seeding identically in both functions for the *fairness* of the Q-vs-QS1 SE comparison? Is the convergence $O(1/\sqrt{M})$ either way? **A.** Reseeding with the same 44 means both measures draw the *same* underlying Z1, Z2 streams, so the SE comparison is on common random numbers — the difference between the two SEs is attributable to the change of measure (the variance of the payoff functional), not to lucky/unlucky draws. That makes the comparison fair (paired). The $O(1/\sqrt{M})$ rate holds under either measure regardless — it is the CLT and depends only on finite payoff variance — what changes is the *constant* multiplying $1/\sqrt{M}$.
15. Using Scenario-1 numbers ($\sigma_1=\sigma_2=0.2$, $\rho=0.9999$), compute $\sigma^* = \sqrt{(\sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2)}$ and explain why this near-zero value is the reason the QS1 SE on line 75 is so small. **A.** $\sigma^2 = 0.04 + 0.04 - 2(0.9999)(0.2)(0.2) = 0.08 - 0.079992 = 8e-6$, so $\sigma^* \approx 0.00283$. With $\rho \approx 1$ the two assets move almost identically, so the basket spread S1-S2 (and the payoff's randomness after switching numéraire to S1) is governed by this tiny σ^* . Under QS1 the estimator's variance scales with σ^2 , so it nearly collapses to a deterministic value — the payoff has almost no variance — making the QS1 SE on line 75 orders of magnitude smaller than the Q SE. This is the variance-reduction win of the S1-numéraire formulation for highly correlated baskets.

Assignment set 1

Open the named file and answer at the line/function level. Point to the exact line, don't paraphrase the report.

1. **2bCF (2).py:26** — `fk = (2/(b-a)) * Im{ char(wk) * exp(-i wk a) }`. Explain term by term where this comes from. Why the imaginary part? Why the $2/(b-a)$ prefactor (and not $1/(b-a)$)? Why does k start at 1 (:22) with no $k=0$ term? **A.** This is the Fourier-*sine* coefficient. Expanding f in $\{\sin(k\pi(x-a)/(b-a))\}$ on $[a,b]$, the coefficient is $f_k = (2/(b-a)) \int_a^b f(x) \sin(\omega_k(x-a)) dx$ with $\omega_k = k\pi/(b-a)$. Writing $\sin(\omega_k(x-a)) = \text{Im}\{e^{i\omega_k(x-a)}\}$ and recognising $\int f(x) e^{i\omega_k x} dx = \phi(\omega_k)$ (the characteristic function), the integral becomes $\text{Im}\{\phi(\omega_k) e^{-i\omega_k a}\}$. So the Im is because the basis is sine (the odd part), $\text{char}(wk) = \phi(\omega_k)$ is the CF, and $\exp(-i \omega_k a)$ is the phase shift to the $[a,b]$ origin. The $2/(b-a)$ is the standard L^2 -normalisation of sine series (not $1/(b-a)$, which is the cosine $k=0$ average term). k starts at 1 because $\sin(0)=0$, so there is no $k=0$ sine mode.
2. **2bCF (2).py:30-31** — the reconstruction loop. Write out the sum being evaluated and say why it approximates $f(x)$ on $[a,b]$. What sets the accuracy: N , or the range $[a,b]=[-10,10]$ (:4-5)? **A.** It evaluates $f_{\text{recovered}}(x) = \sum_{k=1}^N f_k \sin(k\pi(x-a)/(b-a))$, the truncated Fourier-sine partial sum. It approximates f because $\{\sin(k\pi(x-a)/(b-a))\}$ is an orthogonal basis of $L^2[a,b]$ and f_k are the projections; truncating at N keeps the N lowest modes. Both control accuracy but differently: the *truncation* $[a,b]$ must be wide enough to contain essentially all the mass of f (so the series error from the tails is negligible — here $[-10,10]$ is $\sim 10\sigma$ for $N(0,1)$, more than enough), and given a good range, increasing N drives the series-truncation error down (exponentially for this smooth density). With $[-10,10]$ fixed and generous, N is the binding accuracy knob.
3. **2dCF (2).py — function phi_BS** — it returns $\exp(i w \mu_y - 0.5 \text{var}_y w^2)$ with $\mu_y = x + (r - 0.5 \sigma^2) T$. Why is this the characteristic function of $y = \ln(S_T/K)$ and not of $\ln(S_T)$? Where does $x = x_0 = \log(S_0/K)$ enter? **A.** Under Q, $\ln(S_T) = \ln(S_0) + (r - \frac{1}{2}\sigma^2)T + \sigma\sqrt{T} Z$ is normal with mean $\ln(S_0) + (r - \frac{1}{2}\sigma^2)T$ and variance $\sigma^2 T$. Define $y = \ln(S_T/K) = \ln(S_T) - \ln K$; this just shifts the mean by $-\ln K$, giving mean $\ln(S_0/K) + (r - \frac{1}{2}\sigma^2)T = x_0 + (r - \frac{1}{2}\sigma^2)T$. The CF of a normal $N(\mu_y, \sigma^2 T)$ is $\exp(iw\mu_y - \frac{1}{2}\sigma^2 T w^2)$, exactly the return value. So it is the CF of $y = \ln(S_T/K)$ (centred on the strike) precisely because μ_y carries $x = x_0 = \ln(S_0/K)$; that x is the only place the spot/strike ratio enters, and it is what makes the expansion be of the log-*moneyness* rather than the raw log-price.
4. **2dCF (2).py — cumulant range block** — `a = c1 - L*sqrt(c2 + sqrt(c4))` with $c1 = x_0 + \mu T$, $c2 = T \sigma^2$, $c4 = 0$, $L = 10$. Derive $c1$ and $c2$ as the first and second cumulants of the log-price. Why is $c4 = 0$ legitimate for Black-Scholes? What breaks if L is too small? **A.** For $y = \ln(S_T/K) \sim N(x_0 + (r - \frac{1}{2}\sigma^2)T, \sigma^2 T)$, the first cumulant (mean) is $c1 = x_0 + \mu T$ with

$\mu = r - \frac{1}{2}\sigma^2$ (line: $\mu = (r - 0.5\sigma^2)$), and the second cumulant (variance) is $c2 = \sigma^2 T$. A normal distribution has *all* cumulants of order ≥ 3 equal to zero, so $c4 = 0$ is exact for Black-Scholes — that is legitimate, not an approximation. The Fang-Oosterlee rule [$c1 \pm L\sqrt{(c2+v/c4)}$] then reduces to $c1 \pm L\sigma\sqrt{T}$, i.e. ± 10 standard deviations. If L is too small the interval $[a, b]$ does not contain the effective support of the density/payoff, the truncated sine/cosine series misses mass, and the COS price acquires a large, non-decaying *truncation* error that no increase in N can fix.

5. **2dCF (2).py — functions phi_k and chi_k** — note the naming clash: the function called `phi_k` is the report's `psi_k` (integral of `sin`), and `chi_k` matches the report's `chi_k` (integral of `e^y sin`). Reconcile the code with your 2c derivation and confirm `V_k_put` pairs them correctly. Why does the function name `phi_k` collide awkwardly with the characteristic function `phi`? **A.** The code's `phi_k` computes $(1/\omega_k)[\cos(\omega_k(c-a)) - \cos(\omega_k(d-a))]$, which is $\int_c^d \sin(\omega_k(y-a)) dy$ — that is the report's ψ_k (the bare-sine integral). The code's `chi_k` computes $\int_c^d e^y \sin(\omega_k(y-a)) dy$ via the standard $e^y(\sin - \omega_k \cos)/(1+\omega_k^2)$ antiderivative — that matches the report's χ_k . For a put, intrinsic $K - S = K(1 - e^y)$ on $y < 0$, so $V_k \propto \int (K - Ke^y)\sin(\dots) = K(\psi_k - \chi_k)$; `V_k_put` returns $(2K/(b-a))(-\chi_k + \psi_k)$, i.e. $K(\psi_k - \chi_k)$ — correctly paired. The name `phi_k` is unfortunate because `phi/phi_BS` already denote the characteristic function; the local `phi_k` is a payoff-integral, conceptually unrelated, so the clash is purely cosmetic naming, not a bug. (*confirm this matches your own reasoning/observation*)
6. **2dCF (2).py — V_k_put** — returns $(2K/(b-a)) * (-\chi_k(\dots, a, 0) + \psi_k(\dots, a, 0))$. Why are the integration limits $(a, 0)$ and not (a, b) ? Tie this to the put payoff being zero for $y > 0$. **A.** The put payoff in log-moneyness is $K \max(1 - e^y, 0)$, which is positive only for $y < 0$ (i.e. $S_T < K$) and identically zero for $y > 0$. So the coefficient integral $\int_a^b \text{payoff} \cdot \sin = \int_a^0 K(1 - e^y)\sin$ — the $[0, b]$ part contributes nothing. The limits $(a, 0)$ simply restrict to the region where the put has value; integrating over (a, b) would add a zero contribution but the analytic ψ_k, χ_k are written for the active region, so 0 is the correct upper limit.
7. **2dCF (2).py — sin_put_price** — final line is `exp(-rT) * dot(cf_part, V_k)`. Why e^{-rT} and not $e^{-r \Delta t}$ with a different Δt ? Why is the price a dot product of the CF part with the payoff coefficients (what theorem makes the product of two functions become a product of their sine coefficients)? **A.** It is a one-shot European option to maturity T with no intermediate steps, so the discount is over the whole horizon, e^{-rT} ; there is no time grid hence no Δt . The price is $e^{-rT}E[\text{payoff}] = e^{-rT}\int \text{payoff}(y) f(y) dy$. Expanding both f and the payoff in the same orthogonal sine basis on $[a, b]$ and applying Parseval/Plancherel, the integral of the product equals the (weighted) sum of products of their coefficients: $\int f \cdot \text{payoff} \approx \sum_k \text{Im}\{\phi(\omega_k)e^{-i\omega_k a}\} V_k$. That is exactly `dot(cf_part, V_k)` — Parseval's theorem turns the function inner product into a coefficient dot product.
8. **2eCF (2).py:46** — return $(2/\pi) * (\text{im}/\text{ks}) @ (1 - \text{cosine}).T$. Derive this from integrating the sine density term by term. Where do the $1/k$ factor and the $2/\pi$ prefactor come from, and why is the integrated `sin` term $(1 - \cos(\dots))$? **A.** The CDF is $F(z) = \int_a^z f(x)dx$. With $f(x) \approx \sum_k f_k \sin(\omega_k(x-a))$ and $f_k = (2/(b-a)) \text{Im}\{\phi(\omega_k)e^{-i\omega_k a}\}$ ($= \text{im}$ in code), integrate each mode: $\int_a^z \sin(\omega_k(x-a))dx = (1/\omega_k)[1 - \cos(\omega_k(z-a))]$. The lower-limit $\cos(0)=1$ produces the 1 in $(1 - \cos(\dots))$. Now $(2/(b-a)) \cdot (1/\omega_k) = (2/(b-a)) \cdot (b-a)/(k\pi) = 2/(k\pi)$, so the $(b-a)$ cancels and leaves the $2/\pi$ prefactor and the $1/k$ factor (`im/ks`). Hence $F(z) = (2/\pi) \sum_k (\text{im}_k/k)(1 - \cos(k\pi(z-a)/(b-a)))$, exactly the matrix expression $(2/\pi)(\text{im}/\text{ks}) @ (1 - \text{cosine}).T$.
9. **2eCF (2).py:14-15** — fixed range $a=-2, b=2$. For $\ln(S_T/S_0) \sim N(\mu, \sigma_{\text{aux}}^2)$ with $\mu \approx -0.015, \sigma_{\text{aux}} = 0.3$, how many standard deviations is $[-2, 2]$? Is that wide enough? Why does the 2e error plateau at $3.67e-11$ rather than reaching $1e-16$ as 2d does? **A.** Here $\sigma_{\text{aux}} = \sigma\sqrt{T} = 0.3 \cdot \sqrt{1} = 0.3$ and $\mu \approx (r - \frac{1}{2}\sigma^2)T = 0.03 - 0.045 = -0.015$. The interval reaches $(2 - (-0.015))/0.3 \approx 6.7 \sigma$ above and $(-2 - (-0.015))/0.3 \approx -6.6 \sigma$ below the mean — roughly $\pm 6.7\sigma$, which is wide enough that the Gaussian tail mass outside is $\sim 10^{-11}$. That tail mass is precisely the floor: the CDF reconstruction can be no more accurate than the density mass it ignores beyond $[a, b]$, so 2e plateaus at $\approx 3.67e-11$ (the truncation tail), whereas 2d's *price* is an integral against a payoff that decays, letting it reach machine precision. The plateau is

a domain-truncation error, not a series-truncation error — increasing N will not push it below the tail. (*confirm this matches your own observation*)

10. **3aCF (2).py:26** — $S_T = S_0 * \exp(\text{drift} * T + \text{sigma1} * W_t[:, \text{None}] + \text{sigma2} * B_i)$ with $\text{drift} = r - 0.5 * (\text{sigma1}^2 + \text{sigma2}^2)$. Derive this exact terminal law from the SDE (your 3a Ito derivation). Why is $0.5 * (\text{sigma1}^2 + \text{sigma2}^2)$ the right convexity correction, and why is no time-stepping needed? **A.** The asset follows $dS/S = r dt + \sigma_1 dW + \sigma_2 dB$ with W the common factor and B the idiosyncratic factor, independent. Applying Itô to $\ln S$: $d \ln S = (r - \frac{1}{2}(\sigma_1^2 + \sigma_2^2))dt + \sigma_1 dW + \sigma_2 dB$, since the total quadratic variation of the log is $(\sigma_1^2 + \sigma_2^2)dt$ (W and B independent, so no cross term). Integrating to T gives $\ln(S_T/S_0) = (r - \frac{1}{2}(\sigma_1^2 + \sigma_2^2))T + \sigma_1 W_T + \sigma_2 B_T$ with $W_T, B_T \sim N(0, T)$. So $\frac{1}{2}(\sigma_1^2 + \sigma_2^2)$ is exactly the Itô convexity correction for the *combined* variance. Because the drift and diffusion are constant, the SDE has a closed-form lognormal solution at T — Itô integrates exactly — so no Euler time-stepping is needed; one draw of (W_T, B_i) gives S_T exactly. (Note: $W_t[:, \text{None}]$ broadcasts the single common shock across all 1000 stocks; B_i is the per-stock idiosyncratic shock.)
11. **3aCF (2).py:42** — $q = \text{np.quantile}(L_T, 1 - \alpha)$ with $\alpha = 1 - 0.95$. Given the assignment's definition $P(L_T \geq q_\alpha) \leq \alpha$, justify taking the 0.95 empirical quantile. What interpolation does np.quantile use, and could that matter for a discrete L_T ? **A.** $\alpha = 1 - 0.95 = 0.05$, so $1 - \alpha = 0.95$ and $\text{np.quantile}(L_T, 0.95)$ returns the value q with $P(L_T \leq q) \approx 0.95$, equivalently $P(L_T > q) \approx 0.05 = \alpha$ — the 95% VaR-style quantile required. np.quantile defaults to *linear* interpolation between order statistics. For a discrete integer-valued L_T (it is a count, $\sum 1\{S_i > K\}$) the true quantile is an integer, and linear interpolation can return a non-integer between two adjacent counts; this introduces a small interpolation artefact, though with 500k paths it is negligible. A `method='lower'/'higher'` choice would respect the $\leq \alpha$ inequality strictly. (*confirm this matches your own reasoning*)
12. **3bCF (2).py:53** — $\text{ratio} = \exp(-(\mu_{\text{shift}}/T) * W_q + \mu_{\text{shiftsq}})$ with $\mu_{\text{shiftsq}} = \mu_{\text{shift}}^2/(2T)$. Derive this p/q from two Gaussians $p \sim N(0, T)$, $q \sim N(1.5, T)$ and confirm it equals $\exp(-1.5 w + 1.125)$ at $T=1$. Why is only W reweighted and not the B_i (:44)? **A.** The likelihood ratio is $p(w)/q(w) = \exp(-w^2/2T) / \exp(-(w-\mu)^2/2T) = \exp([-w^2 + (w-\mu)^2]/2T) = \exp((-2\mu w + \mu^2)/2T) = \exp(-(\mu/T)w + \mu^2/(2T))$ with $\mu = \mu_{\text{shift}} = 1.5$. That is exactly $\exp(-(\mu_{\text{shift}}/T)W_q + \mu_{\text{shiftsq}})$. At $T=1$, $\mu^2/(2T) = 2.25/2 = 1.125$, so $\text{ratio} = \exp(-1.5 w + 1.125)$, confirming the comment. Only W is reweighted because the importance shift was applied only to the *common* factor W (sampled from $N(1.5, T)$ on line 43); the idiosyncratic B_i are still drawn from their original $N(0, T)$ (line 44), so their density is unchanged and contributes a factor of 1 to p/q . Shifting only W is enough because the rare event (many $S_i > K$) is driven by the common factor.
13. **3cCF (2).py:54** — $\text{ratio} = \sqrt{2} * \exp((-W_q^2 - 3 W_q + 2.25)/(4T))$. Derive this from $p \sim N(0, T)$, $q \sim N(1.5, 2T)$. Where does the $\sqrt{2}$ prefactor come from? Explain why this mean+var shift is *less* efficient for the 95% quantile than the pure mean shift in 3b. **A.** $p(w) = (2\pi T)^{-1/2} \exp(-w^2/2T)$, $q(w) = (2\pi \cdot 2T)^{-1/2} \exp(-(w-1.5)^2/(2 \cdot 2T))$. Then $p/q = \sqrt{(2T/T)} \cdot \exp(-w^2/2T + (w-1.5)^2/4T) = \sqrt{2} \cdot \exp([-2w^2 + (w-1.5)^2]/4T) = \sqrt{2} \cdot \exp((-w^2 - 3w + 2.25)/4T)$, exactly the code. The $\sqrt{2}$ is the ratio of the normalising constants $\sqrt{(\text{var}_q/\text{var}_p)} = \sqrt{(2T/T)}$. It is less efficient because inflating the variance ($2T$ vs T) makes the likelihood ratio p/q much more dispersed — its tail produces a few paths with very large weights, raising the *variance of the weights* and hence of the weighted quantile estimator. The pure mean shift of 3b keeps the variance matched, so the weights are bounded and well-behaved; for a quantile (not a mean) of a common-factor-driven rare event, the mean shift alone already moves mass into the tail without the weight blow-up, so it wins. (*confirm this matches your own observation*)
14. **3bCF (2).py:20-34 / 3cCF (2).py:20-34** — `weighted_quantile: cumsum(w_sorted)/sum(w_sorted)` then `searchsorted(cdf, q)`. Why must you sort by values first and carry the weights along? Why does 3a use `np.quantile` but 3b/3c need this custom weighted version — and does that estimator mismatch bias the efficiency comparison against 3a? **A.** A quantile is read off the CDF, and the empirical CDF is `cumsum` of weights ordered by the value axis; so

you must `argsort(values)` and permute the weights by the *same* sorter (lines 25–27) to keep each weight attached to its observation. `cumsum(w_sorted)/sum(w_sorted)` is then the weighted empirical CDF and `searchsorted(cdf, q)` finds the first value whose cumulative mass reaches q . 3a is plain MC under the true measure p , where every path has equal weight $1/N$, so `np.quantile` (equal-weight) is correct. 3b/3c sample under q , so each path carries its likelihood ratio p/q as a weight, and the unweighted `np.quantile` would be biased — the weighted version restores an unbiased estimate of the p -quantile. The estimator mismatch does *not* bias the comparison: both target the same population quantile of L_T under p ; what differs is the variance of the estimator, which is exactly the efficiency being compared. (One caveat: with linear interpolation in 3a vs `searchsorted` index lookup in 3b/3c, the tiny discretisation conventions differ, but at 500k paths this is immaterial.) (*confirm this matches your own reasoning*)

Exercise set 2

Files: - `assignments/exercise-set-2/exercisel_def (2).py` (LSM / Longstaff-Schwartz) - `assignments/exercise-set-2/exercise2_def (2).py` (FTCS finite differences)

Exercise 1 — LSM

1. **exercisel_def (2).py:96-127** — walk the backward LSM loop out loud. Which line builds the discounted continuation target Y , which applies the ITM mask, which fits the regression, which makes the exercise decision, and which zeroes the later cash flows? **A.** The loop runs i from $T-2$ down to 0. Line 101 `Y = C[:, i+1:] @ discount_factors` builds the discounted continuation target (all realised future cash flows discounted back to $i+1$). Line 102 `itm_indexes = S[:, i+1] < K` is the ITM mask, applied on lines 103–104 to get `X_itm`, `Y_itm`. Lines 107–108 fit the degree-2 regression (`model.fit(X_itm, Y_itm)`). The exercise decision is lines 122–124: `if exercise[j] > continuation[j]` set `C[j, i] = exercise[j]`. Line 125 `C[j, i+1:] = 0.00` zeroes the later cash flows on exercised paths.
2. **:99-101** — Y is the sum of all future cash flows discounted by $\exp(-r \cdot dt \cdot (u-i))$. Why discount to time $i+1$ (not to 0)? What would change in the comparison at :117-120 if you discounted to 0 instead? **A.** `u = arange(i+1, ncols)` and the factor is $\exp(-r \cdot dt \cdot (u-i))$, which discounts each future cash flow to time i (one step ahead of i). The exercise decision at step $i+1$ compares the *intrinsic value at $i+1$* against the *continuation value at $i+1$* , so both sides must be expressed at the same date. The regression target Y and the exercise payoff `exercise = K - S[:, i+1]` must live at the same time point; discounting Y to 0 instead would put continuation at time 0 while intrinsic stays at $i+1$, making the comparison on lines 117–124 mix two time points and corrupt the early-exercise decision. (The absolute level only needs to be consistent on both sides of the inequality.) (*confirm this matches your own reasoning*)
3. **:102-104** — the ITM mask `S[:, i+1] < K`. Why does Longstaff-Schwartz regress *only* on in-the-money paths? What bias appears if you include OTM paths? **A.** Early exercise is only ever relevant where the option is in the money (a put with $S \geq K$ has zero intrinsic, so you would never exercise). Restricting the regression to ITM paths concentrates the basis functions on the region where the exercise boundary actually lives, giving a far better local fit of the continuation value near the boundary. Including OTM paths forces the low-degree polynomial to also fit the flat zero-intrinsic region, distorting the fitted continuation function precisely near the boundary, which biases the exercise decision and typically *underprices* the option. This is the original Longstaff-Schwartz (2001) prescription.
4. **:107-112** — degree-2 monomial pipeline; you reported $E[Y|X] = -1.06999 + 2.98341 X - 1.81358 X^2$ at $t=2$. Reproduce how `a0`, `a1`, `a2` are fit and what X , Y are at that step. **A.** At $i=1$ (so step $t=i+1=2$), X is `S[:, 2]` masked to ITM paths (line 98 builds `X = S[:, i+1] · 1{S<K}` reshaped to a column; `X_itm` keeps the ITM rows), and Y is `Y_itm`, the discounted realised cash flows from $t=3$ back to $t=2$ (`C[:, 2:] @ exp(-r·dt)`). The pipeline `PolynomialFeatures(degree=2, include_bias=False)` builds columns $[X, X^2]$, and `LinearRegression(fit_intercept=True)` does ordinary least squares, returning `intercept_ = a0` and `coef_ = (a1, a2)` (lines 111–112).

Those OLS coefficients are the reported $-1.06999 + 2.98341 X - 1.81358 X^2$, the conditional-expectation estimate $\hat{E}[Y|X]$. (*confirm the numbers match your printout*)

5. **:122-127** — the exercise rule sets $C[j,i]$ to the intrinsic value and zeroes $C[j,i+1:]$. Why must all later cash flows be zeroed once you exercise? What does the stopping matrix at :131 then represent? **A.** An American option is exercised at most *once*; once you exercise at i , the contract is dead and there can be no later cash flow on that path. Zeroing $C[j,i+1:]$ enforces a single realised cash flow per path, so summing the row gives the path's payoff without double counting. The stopping matrix at line 131 ($\text{np.where}(C!=0,1,0)$) then has exactly one 1 per exercised path marking its optimal stopping time — it is the discrete exercise/stopping policy.
6. **:147-156** — American vs European price. Why does the European price use only the terminal column C_{european} while the American price sums each path's single realized cash flow? **A.** The European option can only be exercised at T , so its cash flow is the terminal payoff $C_{\text{european}} = \max(K - S_T, 0)$ (saved on line 93 before the loop overwrites C), discounted once over the full horizon: $\text{mean}(C_{\text{european}} \cdot \exp(-r \cdot dt \cdot T))$ (line 156). The American option may exercise at any step, and after the backward loop each path's row of C holds its single realised cash flow at its optimal time; line 150 discounts each by $\exp(-r \cdot dt \cdot t)$ for its own t and sums per path, then averages (lines 150–151). Different treatment because the two have different exercise rights.
7. **:177-178** — antithetic paths (Z then $-Z$). State the covariance argument for why this reduces variance for this payoff. Does the s.e. $\text{std}(\text{cashflow})/\sqrt{N}$ at :258 remain correct when the N draws are antithetically paired (not independent)? **A.** With antithetics the estimator averages $\frac{1}{2}(f(Z) + f(-Z))$. Its variance is $\frac{1}{4}[\text{Var } f(Z) + \text{Var } f(-Z) + 2\text{Cov}(f(Z), f(-Z))]$ $= \frac{1}{2} \text{Var } f(Z) (1 + \rho)$ with $\rho = \text{Corr}(f(Z), f(-Z))$. For a monotone payoff like a put (decreasing in S , hence in Z), $f(Z)$ and $f(-Z)$ are negatively correlated, $\rho < 0$, so the bracket is below $\text{Var } f(Z)$: variance is reduced. The caveat on line 258: $\text{std}(\text{cashflow})/\sqrt{N}$ treats the N draws as i.i.d., but the antithetic pairs are *dependent*, so this formula is biased (it ignores the within-pair negative covariance and typically *overstates* the true SE). The correct SE computes the std over the $N/2$ *pair averages* divided by $\sqrt{(N/2)}$. So the reported s.e. is conservative/incorrect in form. (*confirm — this is a real subtlety Fang may push on*)
8. **:185-191** — `laguerre_basis` uses $x = S/K$ and weights $\exp(-x/2)$. Why normalise by K , and what numerical problem (underflow) does $\exp(-x/2)$ with $x=S/K$ avoid that raw monomials in S would hit for $S \in [36, 44]$? **A.** Normalising $x = S/K$ makes the regressor $O(1)$ (around 0.9–1.1 for $S \approx 36$ – 44 , $K=40$) instead of $O(40)$; raw monomials S, S^2, S^3 in absolute price would span ~ 36 to $\sim 85k$, producing a badly conditioned (near-collinear, huge-dynamic-range) design matrix and ill-conditioned normal equations. The Laguerre weight $\exp(-x/2)$ with the small normalised x keeps the basis bounded and well-scaled; with raw S the weight $\exp(-S/2)$ would underflow to $\sim \exp(-18) \approx 10^{-8}$ (or smaller), collapsing the basis to numerical zero. Normalising by K is exactly the rescaling Longstaff-Schwartz use to keep the regression well-conditioned.
9. **:257 vs exercise2_def (2).py:223** — one uses $\text{abs}(\text{american-european})$, the other the *signed* difference for the early-exercise value. Which matches the paper's "early-exercise value" column, and can LSM-American ever dip below European by Monte-Carlo noise? **A.** The signed difference $\text{american} - \text{european}$ (`exercise2_def (2).py:223`) matches the paper's "early-exercise premium" column — early-exercise value is by definition $\text{American} - \text{European} \geq 0$ and its sign carries information. The $\text{abs}(\dots)$ on `exercisel_def (2).py:257` discards that sign. The signed form can go slightly negative: LSM is a *suboptimal* (low-biased) estimator of the American price (the regression-based exercise rule is never better than optimal, and any policy is suboptimal in-sample noise aside), and on top of that Monte-Carlo noise can push the simulated American below the closed-form European, giving a small negative premium. The abs hides exactly this diagnostic, so the signed difference is the honest column. (*confirm this matches what you observed in the deep-OTM/short-T rows*)

Exercise 2 — FTCS

10. **exercise2_def (2).py:35-40** — derive the explicit evolution matrix $F = (1 - r k) I + 0.5 k \sigma^2 / h^2 \cdot T2 + k (r - 0.5 \sigma^2) / (2h) \cdot T1$ from the Black-Scholes PDE written in $m = \log S$. Identify which term is the reaction, which the diffusion, which the advection. **A.** Writing $m = \ln S$, the BS PDE in *time-to-maturity* form is $\partial U / \partial \tau = \frac{1}{2} \sigma^2 \partial^2 U / \partial m^2 + (r - \frac{1}{2} \sigma^2) \partial U / \partial m - rU$. Discretise with forward Euler in time (step k) and central differences in m (step h): $(U^{n+1} - U^n) / k = \frac{1}{2} \sigma^2 / h^2 (T2 U^n) + (r - \frac{1}{2} \sigma^2) / (2h) (T1 U^n) - r U^n$, where $T2 = \text{tridiag}(1, -2, 1)$ is the second-difference and $T1 = \text{tridiag}(-1, 0, 1)$ the central first-difference. Solving for U^{n+1} gives $U^{n+1} = [(1 - rk)I + \frac{1}{2} k \sigma^2 / h^2 T2 + k(r - \frac{1}{2} \sigma^2) / (2h) T1] U^n = F U^n$. Identification: $(1 - rk)I$ is the **reaction** (the $-rU$ discount term), $\frac{1}{2} k \sigma^2 / h^2 T2$ is the **diffusion**, and $k(r - \frac{1}{2} \sigma^2) / (2h) T1$ is the **advection** (drift of log-price).
11. **:29-33** — $L = 2 \log K + 2$, $N_x = 1000$, $N_t = 40000 \cdot T$. Why center the grid at $\log K$ (:43)? What is the analogy between this domain truncation and the COS $[a, b]$ range bug you hit in Assignment 2? **A.** The payoff kink and the early-exercise boundary both sit near $S=K$, i.e. $m = \ln K$, so centring the grid there (`mvec += np.log(K)`, line 43) puts the finest action and the interpolation point $\ln S_0$ in the well-resolved interior, keeping the truncated boundaries far from the region of interest. The analogy: in both methods you replace an integral/PDE on $(-\infty, \infty)$ by one on a finite $[a, b]$; if $[a, b]$ (here L) is too narrow it cuts off mass/payoff support and the boundary error dominates and *will not* vanish as you refine N (COS) or N_x, N_t (FD). In Assignment 2 the COS $[a, b]$ collapsed at short maturity (small $c2 = \sigma^2 T$); here $L = 2 \ln K + 2$ is fixed and generous, the FD analogue of choosing $L \sqrt{c2 + \sqrt{c4}}$ wide enough. (*confirm this matches your own reasoning*)
12. **:45-46 and :51** — initial condition $\max(K - e^m, 0)$ and the boundary term $p[0]$ scaled by $K \cdot \exp(-r \cdot \text{time2mat})$. What Dirichlet boundary condition does $p[0]$ enforce at the deep-ITM edge, and why is no correction added at the high- S boundary? **A.** The initial condition (line 46, $\tau=0$) is the put payoff $\max(K - e^m, 0)$. At the lower edge $m \rightarrow -\infty$ (deep ITM, $S \rightarrow 0$) the put value tends to $K e^{-r\tau} - S \approx K e^{-r\tau}$ (discounted strike). The central-difference stencil at the first interior node references the ghost node just below the grid, whose Dirichlet value is $K e^{-r\tau}$; $p[0]$ (line 51) injects exactly that boundary value, scaled by the matching stencil coefficient $(\frac{1}{2} k \sigma^2 / h^2 - k(r - \frac{1}{2} \sigma^2) / (2h))$ and $K \cdot \exp(-r \cdot \text{time2mat})$. At the high- S edge ($m \rightarrow \infty$) the put value $\rightarrow 0$, so the Dirichlet value there is zero and no correction term is needed — p stays zero on that side.
13. **:105** — $U = \text{np.maximum}(U, \text{intrinsic})$, “the single line that makes it American”. Why is projecting onto the payoff after each explicit step a valid (operator-splitting / explicit-penalty) treatment of the free boundary, and what accuracy does it cost versus a proper LCP/PSOR solve? **A.** The American put solves a linear complementarity problem: $U \geq g$, $LU \leq 0$, with complementarity. The explicit projection $U \leftarrow \max(U, g)$ after each time step is a Bermudan-style operator splitting: advance the PDE one step, then enforce the constraint $U \geq \text{intrinsic}$ by projecting the violated nodes up to the payoff. Because the time step k is tiny, the splitting error per step is $O(k)$ and the discrete exercise dates approximate continuous exercise. It is valid but only *first-order* accurate in the constraint handling and it smears the free boundary by one grid/time cell. A proper LCP/PSOR solve treats the constraint *implicitly and simultaneously* with the PDE at each step (projected SOR iteration), locating the free boundary more accurately and without the explicit-projection bias — at the cost of an iterative solve per step. (*confirm this matches your own reasoning*)
14. **Stability:** with $N_x=1000$, $N_t=40000 \cdot T$, worst-case $\sigma=0.4$, compute the diffusion number $0.5 \sigma^2 k / h^2$ and check it is $\leq 1/2$. Is the scheme stable? Why might the author have picked N_t so large (stability vs accuracy)? Can the $T1$ advection term cause oscillations even when the diffusion bound holds? **A.** $h = L/N_x$ with $L = 2 \ln 40 + 2 \approx 9.378$, so $h \approx 9.378/1000 \approx 9.38e-3$, $h^2 \approx 8.79e-5$. $k = T/N_t = T/(40000T) = 1/40000 = 2.5e-5$. Then $\frac{1}{2} \sigma^2 k / h^2 = \frac{1}{2} \cdot 0.16 \cdot 2.5e-5 / 8.79e-5 \approx 0.5 \cdot 0.16 \cdot 0.2845 \approx 0.0228$, well under $\frac{1}{2}$. So the explicit FTCS scheme is stable (von Neumann diffusion bound satisfied with large margin). N_t is huge mostly to satisfy this $k \leq h^2 / \sigma^2$ stability constraint — explicit schemes force $k = O(h^2)$, so refining N_x (smaller h) demands quadratically more time steps; accuracy is a secondary beneficiary. The $T1$ advection

(central difference) can produce spurious oscillations when the *cell Péclet number* $|r - \frac{1}{2}\sigma^2| h / \sigma^2$ exceeds ~ 1 , independently of the diffusion bound; here drift is small and h tiny so Péclet $\ll 1$, no oscillation, but in principle yes it can. (*confirm the numbers against your printout*)

15. **:54 / :107** — `np.interp` reads the price at $\log(S_0)$. Your FD-European prices are systematically $\sim 1e-4$ below Black-Scholes (every difference negative). Is the consistent sign due to the explicit scheme, the truncated domain, or the interpolation? Defend your pick. **A.** The consistent negative sign points to the *truncated domain* as the dominant source. Linear `np.interp` on a convex value function biases the read-off *upward* (chord lies above the curve), so interpolation alone would push FD *above* BS — wrong sign. The explicit FTCS truncation error is $O(k) + O(h^2)$ and is essentially unsigned/oscillatory across the parameter grid, so it would not produce a uniform sign on every row. Truncating the domain removes a sliver of probability mass in the tails that the option payoff would have weighted, so the discrete value is systematically a touch *low* everywhere — a one-sided, consistent $\sim 1e-4$ deficit, matching the observation. So I attribute the uniform negative difference to domain truncation, not to the scheme or the interpolation. (*confirm this matches your own observation of the sign on every row*)

Assignment set 2

Files: - assignments/assignment-set-2/Q1_basket_MC (1).py - assignments/assignment-set-2/Q2_moment_matching (1).py - assignments/assignment-set-2/Q3_basket_COS (1).py

Q3 — basket COS (highest-priority: Fang invented COS)

1. **Q3_basket_COS (1).py:22-32** — `chi_coeffs`: $\chi_k = \text{Re}\{\phi(\omega_k) \exp(-i \omega_k a)\}$, $\omega_k = k \pi / (b-a)$, with $\chi_0 *= 0.5$ at :31. Explain term by term where this comes from, and why the $k=0$ term is halved (the primed sum). **A.** These are the *cosine* density coefficients of the COS method. Expanding the density on $[a, b]$ in $\{\cos(k\pi(x-a)/(b-a))\}$, the coefficient is $A_k = (2/(b-a)) \int_a^b f(x) \cos(\omega_k(x-a)) dx$, $\omega_k = k\pi/(b-a)$ (line 29). Writing $\cos(\omega_k(x-a)) = \text{Re}\{e^{i\omega_k(x-a)}\}$ and using $\int f(x) e^{i\omega_k x} dx = \phi(\omega_k)$ gives $A_k \propto \text{Re}\{\phi(\omega_k) e^{-i\omega_k a}\}$ — exactly line 30. (The code folds the $2/(b-a)$ and the payoff normalisation into the final triple product rather than here.) The $k=0$ term is halved (line 31, `chi[0] *= 0.5`) because the COS reconstruction $f(x) \approx \sum' A_k \cos(\dots)$ uses a *primed* sum where the first term carries weight $\frac{1}{2}$ — this is the standard Fourier-cosine convention (the constant mode is the average, counted once, not twice).
2. **:34-54** — `payoff_coeffs_2d` builds $V_{\{k1, k2\}}$ as $C1^T P C2$ by trapezoidal quadrature on a dense payoff grid. Why is there *no* closed-form V_k here, unlike the 1D European call/put (where χ_k, ψ_k are analytic)? What couples the two state variables? **A.** In 1D the payoff $\max(\pm(S-K), 0)$ is a function of a *single* variable and the cosine integral of e^y and 1 against $\cos$ has the closed-form χ_k, ψ_k . Here the payoff is $\max(w_1 g_1(y_1) + w_2 g_2(y_2) - K, 0)$ — the $\max(\cdot, 0)$ and the strike couple y_1 and y_2 *non-separably*: the region where the basket exceeds K is a diagonal half-plane $\{w_1 g_1(y_1) + w_2 g_2(y_2) > K\}$, not a product of intervals, so the 2D integral does not factor into a product of 1D integrals and has no elementary antiderivative. Hence line 49 builds the payoff on a dense grid P and lines 52-54 apply 2D trapezoidal quadrature written as $C1^T P C2$, where $C1, C2$ are the cosine matrices times the trapezoidal weights ($W_y \cdot h$). It is the strike K inside the joint $\max$ that couples the two states.
3. **:56-63** — `price_2dCOS` assembles $e^{-rT} \cdot \chi_1^T V \chi_2$. What makes the 2D integral collapse to this triple product of coefficient vectors, and where does the $\rho=0$ independence assumption enter (the density coefficients factorising $c_{\{k1, k2\}} = \chi_{\{k1\}} \chi_{\{k2\}}$)? **A.** The price is $e^{-rT} \iint \text{payoff}(y_1, y_2) f(y_1, y_2) dy_1 dy_2$. Expanding the *joint* density in a 2D cosine series, $f(y_1, y_2) \approx \sum_{\{k1, k2\}} c_{\{k1, k2\}} \cos(\omega_{\{k1\}}(y_1 - a_1)) \cos(\omega_{\{k2\}}(y_2 - a_2))$, and by 2D Parseval the integral becomes $\sum_{\{k1, k2\}} c_{\{k1, k2\}} V_{\{k1, k2\}} = \chi_1^T V \chi_2$ once $c_{\{k1, k2\}}$ factorises. The $\rho=0$ independence is exactly what makes it factorise: with independent marginals the joint density is a product $f(y_1, y_2) = f_1(y_1) f_2(y_2)$, so its 2D cosine coefficients separate $c_{\{k1, k2\}} = \chi_{\{k1\}} \chi_{\{k2\}}$ (the outer product of the two 1D coefficient vectors from `chi_coeffs`). That is why the triple product $\chi_1^T V \chi_2$ — and not a full

coefficient *matrix* recovered from a 2D characteristic function — suffices. (If $\rho \neq 0$ you would need the joint CF and a 2D coefficient matrix.)

4. **:65-71** — `cumulant_range` returns $[k1 - \text{width}, k1 + \text{width}]$ with $\text{width} = L \cdot \sqrt{(|k2| + \sqrt{|k4|})}$, $L=12$. Interpret each of $k1$, $k2$, $\sqrt{k4}$, L . **A.** $k1$ is the first cumulant = mean of the marginal, so the interval is centred on the mean. $k2$ is the second cumulant = variance, so $\sqrt{k2}$ is the standard deviation — the basic width scale. $\sqrt{k4}$ adds the fourth-cumulant (kurtosis/tail) contribution: $k4 > 0$ (heavy tails) widens the interval to capture the extra tail mass that a pure-Gaussian $\sqrt{k2}$ would miss. L is the number of “standard-deviation-like units” of half-width (here $L=12$), the safety multiplier from Fang-Oosterlee; larger L means a wider, safer truncation at the cost of needing more terms N for the same resolution.
5. **:82-83** — `Q3(a)` calls `cumulant_range(mu, sigma^2 T, 0)` per marginal. For a normal log-price, what are $k1$, $k2$, and why is $k4 = 0$? **A.** For the normal log-price $x_i = \ln S_i(T) \sim N(\mu_i, \sigma_i^2 T)$ with $\mu_i = \ln 100 + (r - \frac{1}{2}\sigma_i^2)T$ (lines 76-77), the first cumulant $k1 = \mu_i$ (the mean) and the second cumulant $k2 = \sigma_i^2 T$ (the variance). A Gaussian has *every* cumulant of order ≥ 3 equal to zero, so $k4 = 0$ exactly — there is no excess kurtosis, the tails are exactly Gaussian, and the width reduces to $L \sigma_i / \sqrt{T} = 12 \sigma_i / \sqrt{T}$.
6. **THE BUG (over-prepare this).** **:70** — $\text{width} = L \cdot \sqrt{(|k2| + \sqrt{|k4|})}$. As $T \rightarrow 0$, $k2 = \sigma^2 T \rightarrow 0$, so width collapses and $[a, b]$ becomes too narrow to contain the density/payoff support. Show me the exact line where this happens. **TODO (Bruno):** what symptom did you see and at which T ; and exactly what did you change — L at **:65**, the width formula at **:70**, or the non-negativity clamp at **:127**? Why is $\sqrt{c2 + \sqrt{c4}}$ (not just $\sqrt{c2}$) the right safeguard, and why is a fixed wide interval not a free lunch? **A.** The collapse is on **line 70**: $\text{width} = L \cdot \text{np.sqrt}(\text{abs}(k2) + \text{np.sqrt}(\text{abs}(k4)))$. For the normal legs $k2 = \sigma^2 T$ and $k4 = 0$, so $\text{width} = L \sigma / \sqrt{T}$, which $\rightarrow 0$ as $T \rightarrow 0$; with $k4=0$ there is *no floor* on the width, so $[a, b] = [\mu - L \sigma / \sqrt{T}, \mu + L \sigma / \sqrt{T}]$ shrinks to a point around the mean and no longer brackets the support of the density or, crucially, the payoff kink at the strike. The mechanism: at short T the truncation interval can become narrower than the distance from the mean to $\ln K$, so the diagonal exercise region $\{\text{basket} > K\}$ is partly outside $[a, b]$ and the quadrature on the dense grid (lines 43-49) integrates a payoff grid that misses the in-the-money tail. What one *would* expect to see as the symptom: the COS price stops matching the MC/moment benchmark at small maturities (e.g. T of order 0.05-0.1), the error *fails to fall* as you raise N (because it is a domain-truncation error, not a series error), and the price can even drift or go visibly wrong while the long-maturity prices stay correct. The standard fixes, in order of preference: keep the $\sqrt{c2 + \sqrt{c4}}$ form (do **not** drop to $\sqrt{c2}$) and/or add an explicit floor on the half-width, or raise L (line 65); the cleanest is a width floor so $[a, b]$ always contains $\ln K \pm \text{a few } \sigma$. The $\sqrt{c2 + \sqrt{c4}}$ form is the right safeguard because for heavy-tailed/short-maturity laws the variance $c2$ alone *understates* how far the relevant mass reaches — the $\sqrt{c4}$ term adds the kurtosis/tail contribution and keeps the interval from collapsing when $c2$ is tiny but the tails (or the strike distance) still matter. A fixed wide interval is not a free lunch because COS resolution is $N/(b-a)$: widening $[a, b]$ for a fixed N *coarsens* the cosine grid, so you trade truncation error for series-truncation error and must raise N (more cost) to keep the same accuracy — the cumulant rule is precisely the device that picks $[a, b]$ just wide enough. **(Bruno: confirm which exact change you made — L at line 65, the width formula at line 70, or a floor — and at which T you saw the price break; the analysis above is the correct general justification but the specific number/edit is yours.)**
7. **:117-119** — phi2_b is the CIR-leg characteristic function $\exp(\lambda \cdot i \cdot z / (1 - 2i \cdot z))$, $z = \text{om} \cdot c$. Where does this come from (the zero-dof non-central chi-square law of $S2(T) = c \cdot Z$)? **A.** By the technical hint, the CIR terminal value factors as $S2(T) = c \cdot Z$ with $Z \sim \chi'^2_0(\lambda)$, a non-central chi-square with **zero** degrees of freedom and non-centrality λ (lines 114-116 set c_b , lam_b). The CF of a non-central chi-square $\chi'^2_\nu(\lambda)$ is $\exp(i\lambda t / (1 - 2it)) / (1 - 2it)^{\nu/2}$; with $\nu=0$ the denominator power vanishes, leaving $\phi_Z(t) = \exp(i\lambda t / (1 - 2it))$. Since $S2(T) = cZ$, $\phi_{\{S2\}}(\omega) = \phi_Z(c\omega)$, i.e. with $z = \omega \cdot c$ the CF is $\exp(\lambda \cdot iz / (1 - 2iz))$ — exactly line 119. The zero-dof part (no $(1 - 2it)^{\nu/2}$ factor) is the special structure of the square-root process at this parameterisation.

8. **:122-127** — CIR cumulants $k_1 = c \cdot \lambda$, $k_2 = 4 c^2 \lambda$, $k_4 = 192 c^4 \lambda$, and the range is clamped $a_2 = \max(a_2, 0)$ at **:127**. Why is the clamp needed for the square-root process? **A.** For $Z \sim \chi'^2_0(\lambda)$ the cumulants are $\kappa_n = 2^{n-1} n! \lambda$ (so $\kappa_1=\lambda$, $\kappa_2=4\lambda$, $\kappa_4=192\lambda$), and since $S_2=cZ$ they scale as c^n : $k_1=c\lambda$, $k_2=4c^2\lambda$, $k_4=192c^4\lambda$ (lines 122-124). The cumulant rule $[k_1 \pm L\sqrt{(k_2+\sqrt{k_4})}]$ is *symmetric* about the mean, but $S_2(T)$ is a CIR/square-root level that is **non-negative** by construction — the process cannot go below 0. For the heavily right-skewed CIR law the symmetric lower endpoint $a_2 = k_1 - \text{width}$ can fall below 0, which is outside the support and would waste grid (and put the quadrature on a region of zero density). The clamp $a_2 = \max(a_2, 0)$ (line 127) restricts the truncation interval to the physical support $[0, b_2]$, matching the non-negativity of the square-root process.

Q1 — basket Monte Carlo

9. **Q1_basket_MC (1).py:29 and :116** — $Z_2 = \rho Z_1 + \sqrt{1-\rho^2} Z_c$. Show this is exactly the 2×2 Cholesky factor of the correlation matrix, i.e. that $\text{Corr}(Z_1, Z_2) = \rho$. **A.** $Z_1, Z_c \sim N(0,1)$ independent. $\text{Var}(Z_2) = \rho^2 \text{Var}(Z_1) + (1-\rho^2) \text{Var}(Z_c) = \rho^2 + (1-\rho^2) = 1$, and $\text{Cov}(Z_1, Z_2) = E[Z_1(\rho Z_1 + \sqrt{1-\rho^2} Z_c)] = \rho E[Z_1^2] + \sqrt{1-\rho^2} E[Z_1 Z_c] = \rho \cdot 1 + 0 = \rho$. So $\text{Corr}(Z_1, Z_2) = \rho / (1 \cdot 1) = \rho$. The transform $(Z_1, Z_c) \mapsto (Z_1, Z_2)$ is the lower-triangular matrix $L = \begin{bmatrix} 1 & 0 \\ \rho & \sqrt{1-\rho^2} \end{bmatrix}$, which is precisely the Cholesky factor of $\Sigma = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix} = L L^T$.
10. **:120-121** — Euler step for the CIR leg with full truncation $S_2p = \max(S_2, 0)$ inside both drift and diffusion. **TODO (Bruno):** why full truncation rather than reflection, and what bias does it introduce? Why is the Q-drift $r \cdot S_2$ (not the physical $\kappa(\theta - S_2)$)? **A.** Lines 120-121 apply $S_2p = \max(S_2, 0)$ and step $S_2 = S_2 + r \cdot S_2p \cdot h + \sigma_2 \cdot \sqrt{S_2p} \cdot \sqrt{h} \cdot Z_2$. The Euler discretisation of a square-root diffusion can overshoot below zero in a step, leaving $\sqrt{S_2}$ undefined; **full truncation** (Lord, Koekkoek & van Dijk 2010) substitutes S_2^+ only *inside the coefficients* (drift and diffusion) while letting the state itself go slightly negative, then truncates. It is preferred over **reflection** ($|S_2|$ or $-S_2$) because full truncation is the scheme with the smallest discretisation bias among the fixes in that paper and it does not artificially inject spurious positive drift/variance that reflection creates by bouncing paths off zero; reflection systematically biases the level upward near the boundary. Full truncation still introduces a (downward) bias because zeroing the coefficient kills the local drift/vol whenever a path is at/below zero, but it is the mildest. The Q-drift is $r \cdot S_2$, not the physical mean-reverting $\kappa(\theta - S_2)$, because under the risk-neutral measure S_2 is a *traded asset* whose discounted price $e^{-rt} S_2$ must be a martingale; that forces drift $r \cdot S_2 \, dt$. The physical CIR drift $\kappa(\theta - S_2)$ belongs to a (non-traded) rate/variance factor under the real-world measure — here S_2 is a price, so risk-neutral valuation pins its drift to $r \cdot S_2$. (*confirm this matches your own reasoning on the full-truncation choice and any bias you observed*)
11. **:144-169** — the weak-error test reuses the *same* fine Brownian increments across all coarse grids. Why does reusing increments cleanly isolate the discretization bias from the Monte-Carlo noise? **A.** The total error of a discretised MC price is (discretisation bias) + (statistical/MC noise). By generating one set of fine Brownian increments $dW1f, dW2f$ per chunk and *aggregating* them into each coarse grid (basket_on_grid sums $k = n_{\text{fine}}/n_{\text{steps}}$ fine increments into each coarse step, lines 149-152), every grid is driven by the *same* underlying Brownian path. So when you difference two grids, the MC noise (which depends only on the realised path) cancels in common, and what remains is purely the difference in *discretisation* — the bias due to step size h . The price difference $p(n) - p_{\text{ref}}$ is then an almost noise-free estimate of the weak error, which is why the fitted slope on lines 181-185 cleanly recovers the expected weak order 1.0 for Euler; without common increments the MC noise ($\sim 1/\sqrt{M}$) would swamp the small $O(h)$ bias.

Q2 — moment matching

12. **Q2_moment_matching (1).py:20-29** — `lognormal_cross` returns $E[S_1^j S_2^m]$ via $\exp(\text{mean} + 0.5 \, \text{var})$. Derive this cross raw moment for two correlated log-normals by hand. **A.** Write S_1

$= e^X$, $S2 = e^Y$ with (X,Y) jointly normal: $X \sim N(\mu_X, v_X)$, $Y \sim N(\mu_Y, v_Y)$, $\text{Cov}(X,Y) = c_{\{XY\}}$ (lines 26-28: $\mu_X = \ln S10 + (r - \frac{1}{2}\sigma^2)T$, $v_X = \sigma^2 T$, $c_{\{XY\}} = \rho\sigma_1\sigma_2 T$). Then $S1^j S2^m = e^{jX + mY}$, and $jX + mY$ is normal with mean $j\mu_X + m\mu_Y$ and variance $j^2 v_X + m^2 v_Y + 2jm c_{\{XY\}}$. Using the MGF of a normal, $E[e^N] = \exp(\text{mean} + \frac{1}{2}\text{var})$ for N normal, gives $E[S1^j S2^m] = \exp(j\mu_X + m\mu_Y + \frac{1}{2}(j^2 v_X + m^2 v_Y + 2jm c_{\{XY\}}))$ — exactly line 29.

13. **:39-53** — `cir_S2_rawmoments` uses cumulants $\kappa_n = 2^{n-1} n! \lambda$ then a raw-from-cumulant recursion. Derive the non-central chi-square (zero-dof) cumulants κ_n . **A.** From $\phi_Z(t) = \exp(\lambda \cdot it / (1-2it))$ (zero-dof non-central chi-square, see Q3 Q7), the cumulant generating function is $K(t) = \log E[e^{tZ}] = \lambda t / (1-2t)$ (set $it \rightarrow t$). Expand $1/(1-2t) = \sum_{m \geq 0} (2t)^m$, so $K(t) = \lambda \sum_{m \geq 0} 2^m t^{m+1} = \lambda \sum_{n \geq 1} 2^{n-1} t^n$. The cumulant κ_n is $n!$ times the coefficient of t^n in $K(t)$, hence $\kappa_n = n! \cdot \lambda \cdot 2^{n-1} = 2^{n-1} n! \lambda$ (line 49). The code then turns these cumulants into raw moments by the standard recursion $m_n = \sum_{k=0}^{n-1} C(n-1, k) \kappa_{n-k} m_k$ (lines 51-52), and scales $S2 = cZ$ by c^{k+1} (line 53).
14. **:78-139** — `price_2moments` (log-normal), `price_3moments` (shifted log-normal via skewness inversion with `brentq`), `price_4moments` (Johnson SU via `least_squares`). **TODO (Bruno):** why is each family the natural choice for 2/3/4 matched moments, and why need not matching more moments monotonically reduce the pricing error? **A.** Each family has exactly as many free shape parameters as moments to match, and is the canonical positive-support distribution at that order: (2) a **log-normal** has two parameters $(\mu, \sigma^2) \rightarrow$ matches mean and variance, and is natural because the basket is a positive sum close to log-normal (Black/Lévy moment-matching); skewness/kurtosis are then *implied*, not controlled. (3) a **shifted log-normal** $\gamma + L$ adds a location shift γ , giving three parameters, so it can also match **skewness** — the code inverts the closed-form log-normal skewness $(w+2)\sqrt{w-1} = \gamma 1$ for $w = e^{\{s^2\}}$ with `brentq` (line 99), then sets M, γ from variance and mean. (4) the **Johnson SU** $\xi + \lambda \sinh((Z-\gamma)/6)$ has four parameters $(\xi, \lambda, \delta, \gamma)$, enough to match mean, variance, **skewness and kurtosis**; its (δ, γ) are fitted to the standardized skew/kurtosis by `least_squares` (line 134) because their closed forms are awkward, then (ξ, λ) from mean/variance. Matching more moments need *not* reduce the pricing error monotonically because: (a) the call price is an integral against the *whole* density tail, and the error depends on how well the chosen family's shape matches the true basket density near and beyond the strike — a higher-moment family can fit the body better yet distort the tail that the payoff weights; (b) the moment problem can be ill-conditioned (e.g. the Johnson `least_squares` may land on a shape whose tail differs from the true one even though skew/kurtosis agree); and (c) the basket's true law is not exactly any of these families, so adding a constraint trades one mismatch for another rather than strictly improving. So 3- or 4-moment can occasionally do *worse* than 2-moment on the price even though it matches more moments. (*confirm this matches the non-monotone error pattern you saw in your Q2 table*)